Chapitre 3 :

Mise en forme avec CSS

Le langage HTML permet de définir le squelette et le sens d'un document Web en organisant ses contenus. Cependant, une structure brute ne suffit pas à offrir une expérience utilisateur agréable, moderne et accessible.
C'est ici qu'intervient le CSS qui prend en charge la présentation visuelle des éléments objet de ce chapitre.

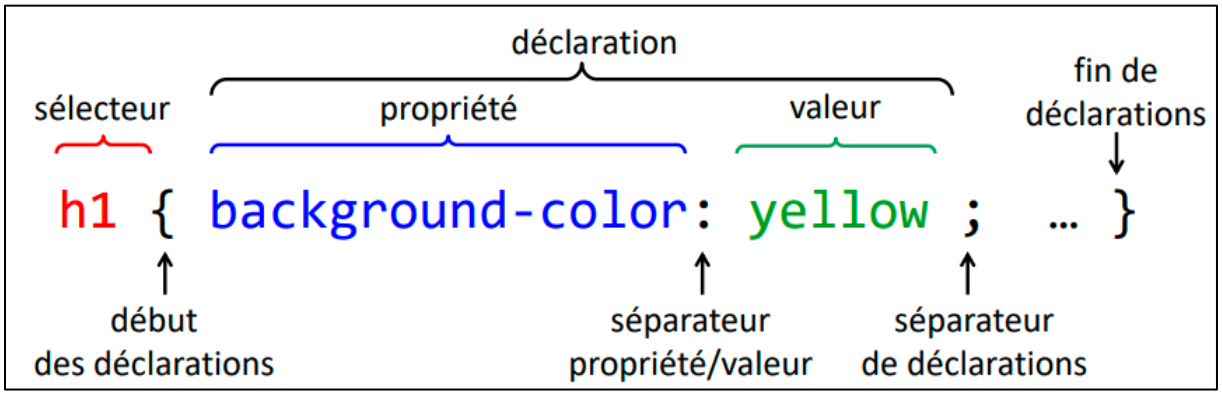
I. Présentation du CSS

Le rôle du CSS (Cascading Style Sheets) est de gérer l'apparence de la page web à travers la définition de style.

Il permet de définir la mise en forme de la page en spécifiant les bordures, la couleur de fond et l'image d'arrière-plan, la police de caractère, l'espace entre les lettres, etc.

Les « effets de style » de la page sont rangés dans des **propriétés** (Exemples : *color* pour indiquer la couleur du texte, *font-size* pour indiquer la taille du texte, etc). Chaque propriété CSS doit avoir une **valeur**.

Ainsi, la structure d'une règle CSS est la suivante :



Le **sélecteur** identifie les éléments HTML sur lesquels la règle CSS s'applique

Exemples de propriétés et de valeurs :

Propriétés	Descriptif	Exemple
color	Fixe la couleur d'avant-plan.	color: blue; color: #0000FF;
font-family	Spécifie la police de caractère à utiliser	font-family : courier;

II. Définition des styles

Il y a trois façons de définir des styles CSS :

1. **Inline** : Directement dans les balises du fichier HTML *via* un attribut **style**
2. **Interne** : Dans l'en-tête de la page
3. **Externe** : Dans une feuille de style totalement séparée du code HTML.

II.1. Définition des styles dans les balises : inline

On peut ajouter un attribut *style* à n'importe quelle balise. On insère le code CSS directement dans cet attribut :

Syntaxe :

```
<balise style="propriété1: valeur; propriété2 : valeur;…"> </balise>
```

Exemple :

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="utf-8" >
<title>Premiers tests du CSS</title>
</head>
<body>
        <h1>Mon  site</h1>
        <p style="color: blue">Bienvenue sur mon site !</p>
        <p>Bonne rentrée à  ISET Charguia</p>
</body>
</html>
```

Résultat :

Cette fois, seul le texte du premier paragraphe dont la balise contient le code CSS, sera coloré en bleu.



II.2. Définition des styles dans l'entête de la page : interne

Cela consiste à insérer le code CSS directement dans une balise **<style>** à l'intérieur de l'en-tête **<head>**.

Syntaxe :

```
<head>
   <style> sélecteur {propriété1 : valeur ;
                      propriété2 : valeur ;
                     …}
   </style>
```

```
</head>
```

Le sélecteur peut être une balise, une classe ou un identifiant qui seront présentés dans la suite.

Exemple :

```
<!DOCTYPE html>
<html lang="fr">
<head>
        <meta charset="utf-8" ><title>Premiers tests du CSS</title>
        <style>
          p {
             color: red;
            }
        </style>
</head>
<body>
                <h1>Mon site</h1>
                <p>Bienvenue sur mon site !</p>
                <p>Bonne rentrée à ISET Charguia</p>
</body>
</html>
```

Résultat :

Les textes de tous les paragraphes sont colorés en rouge.



II.3. Définition des styles dans une feuille de style : externe

La définition des styles dans une feuille de style (CSS = Cascading Style Sheet) permet de :

- Séparer le contenu de la mise en forme
- Modifier l'aspect d'un page ou d'un site sans en modifier le contenu.
- Modifier facilement l'aspect d'une page

Pour ce faire on doit :

1. Définir les styles dans une feuille de style qui est un fichier d'extension **.CSS**
2. Associer le fichier de style ainsi créé au document HTML auquel seront appliqués les styles ainsi définis. Ce lien se fera moyennant la balise **<link>** à placer dans l'entête (**<head>**) selon la syntaxe suivante :

```
<link rel="stylesheet" href="chemin vers le fichier.css">
```

Exemple :

Le fichier de style monStyle.css contient le code :

```
p{ color: blue;}
```

Le fichier HTML associé au fichier de style monStyle.css est :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
      <meta charset="utf-8">
      <link rel="stylesheet" href="monStyle.css">
      <title>Premiers tests du CSS</title>
  </head>
<body>
      <h1>Mon  site</h1>
      <p>Bonjour et bienvenue sur mon site !</p>
      <p>Bonne rentrée à  ISET Charguia</p>
</body>
```

III. Exemples de propriétés CSS

Les propriétés CSS permettent de mettre en forme du texte, des images, des tableaux, des formulaires, etc. Cette partie présente un aperçu de quelques propriétés de texte, de couleur de fond et de bordure des boîtes.

III.1. Propriétés de formatage de texte

Propriété	Valeurs possibles	Description
font-size	- Taille absolue : en pixel (10 px), en centimètres, en millimètres - Taille relative : en pourcentage (100%), en « em » (1.5 em), en « ex » - valeur relative : « x-small », « small », « large », « x-large » ,…	Taille de la police
font-family	Verdana, serif, Arial,…	Police de caractères
font-style	italic, oblique, normal	Style du texte
font-weight	bold, normal	Mise en gras
text-decoration	underline, overline, line-through, blink, none	Soulignement ou décoration
text-align	left, right, center, justify	Alignement
letter-spacing	18px, normal,…	Espacement des lettres
text-transform	capitalize, lowercase, uppercase	Casse du texte
line-height	18px, 120%, normal...	Hauteur de ligne
text-shadow	4 valeurs : `5px 5px 2px blue` (décalage horizontal de l'ombre , décalage vertical de l'ombre, l'adoucissement, couleur de l'ombre)	Ombre du texte

III.2. Propriétés de couleur et fond

Propriété	Valeurs possibles	Description
color	- Nom couleur : red, maroon,… - Code couleur en hexadécimal : #FF5A28 - rgb(val1, val2, val3 : rgb(255,0,0)	couleur
background-color	Nom ou code couleur	Couleur de fond
background-image	Chemin vers l'image : `url('image.png')`	Image de fond

III.3. Propriétés des listes

Propriété	Valeurs possibles et rendu visuel	
list-style-type	***Listes non numérotées***	***Listes numérotées***
	- disc • - circle ○ - square ▪ - ***"chaine"*** ?? - none	***decimal*** 1, 2, 3... ***decimal-leading-zero*** 01, 02, 03... ***lower-alpha*** a, b, c... ***upper-alpha*** A, B, C... ***lower-roman*** i, ii, iii, iv, v ***upper-roman*** I, II, III, IV, V
list-style-image	Chemin vers l'image : `url('image.png')`	
list-style-position	Inside • Le marqueur est à l'intérieur (*inside*) • Le marqueur est à l'intérieur (*inside*)	outside (par défaut) • Le marqueur est à l'extérieur (*outside*) • Le marqueur est à l'extérieur (*outside*)
list-style	Raccourci des propriétés type, position et image ul { list-style: circle inside url('check.png') }	

Exemple :

Code HTML

```
<ul>
  <li>Parcours
    <ol>
      <li>Smart Data</li>
      <li>Network & CLoud</li>
      <li>Software Dévelopment</li>
    </ol>
  </li>
  <li>Option: Python</li>
</ul>
```

CSS

```
ol {
   list-style-type: lower-alpha;
   list-style-image: none;
}
ul {
   list-style: circle inside url('check.png')
}
```

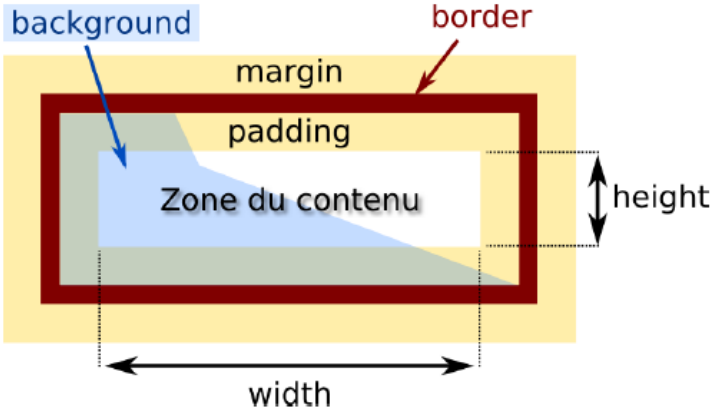
Résultat

- Parcours
  a. Smart Data
  b. Network & CLoud
  c. Software Dévelopment
- Option: Python

III.4. Propriétés des bordures et des boîtes

Tout élément HTML est une boîte composée de 4 couches, de l'intérieur vers l'extérieur :

- Le contenu (content) : Le texte, l'image, etc. (défini par width et height)
- La marge intérieure (padding) : Espace *intérieur* entre le contenu et la bordure.
- La bordure (border) : Le trait qui entoure le padding.
- La marge extérieure (margin) : Espace *extérieur* qui sépare la boîte des autres éléments.



Propriété	Valeurs possibles	Description
border-width	Indiquer une valeur en pixel par exemple	Epaisseur de la bordure
border-color	Nom de couleur ou code couleur	Couleur de la bordure
border-style	none,solid, dotted, dashed, double, groove, ridge, inset, outset	Style de la bordure
border	regroupe les trois propriétés précédentes	Bordure
border-top	border-top-width, border-top-color, border-top-style	Bordure du haut
border-bottom	border-bottom-width, border-bottom-color, border-bottom-style)	Bordure du bas
border-left	border-left-width, border-left-color, border-left-style	Bordure de gauche
border-right	border-right -width, border-right-color, border-right-style	Bordure de droite
border-radius	valeur en pixel par exemple	Arrondi de la bordure
box-shadow	6px 6px 0px black (décalage horizontal, décalage vertical, l'adoucissement, couleur de l'ombre)	Ombre des boîtes

III.5. Aperçu des unités CSS et de leurs cas d'usage

Catégorie	Unité	Référence de calcul	Cas d'usage idéal
Absolue	px	Pixels fixes de l'écran	Bordures fines (1px), ombres, éléments fixes
Relative à la police	rem	Racine (<html>, par défaut 16px)	**Polices de caractères**, padding, margin
	em	Élément parent direct	Composants autonomes (boutons, badges réutilisables)
Relative au conteneur	%	Largeur/Hauteur du parent direct	Largeurs de colonnes, grilles fluides
Relative à l'écran	vw / vh	Largeur/Hauteur de la fenêtre	Sections plein écran
	dvh	Hauteur dynamique (avec/sans barre URL mobile)	Poses plein écran réactives sur smartphone

IV. Les sélecteurs CSS simples

Les sélecteurs définissent les éléments sur lesquels s'applique un ensemble de règles CSS.
Les sélecteurs simples sont les sélecteurs de type, d'identifiant et de classe.

IV.1. Les sélecteurs de type (balise)

Dans une feuille de style on peut indiquer les noms de balises dont on veut modifier l'apparence, et ce, selon la syntaxe suivante :

Syntaxe :

```
balise1 {
     propriete1: valeur1;
     propriete2: valeur2;
}
balise2 {
     propriete1: valeur1;
     propriete2: valeur2;
     propriete3: valeur3;
     propriete4: valeur4;
}
balise3 {
     propriete1: valeur1;
}
```

Exemple :

Soit la feuille de style monStyle.css suivante :

```
p{
        font-size : 15px;
        font-family : Verdana;
        color : black;
        background-color : white;
}
h1{
        font-size : 200%;
        font-family : Arial;
        color : navy;
        text-align: center;
}
```

Résultat :

- Les propriétés de style définies pour la balise **<p>** sont appliquées à tout texte écrit dans une balise **p**.
- Les propriétés de style définies pour la balise **<h1>** sont appliquées à tout texte écrit dans une balise **h1.**

On peut appliquer le même style à plusieurs balises, dans ce cas, on écrit dans le fichier de style :

Syntaxe :

```
Balise1, balise2,… baliseN {
      propriete1: valeur1;
      propriete2: valeur2;
      propriete3: valeur3;
}
```

Exemple :

```
h3, em {
        font-family : "Times New Roman";
        color : navy;
}
```

IV.2. Les sélecteurs d'identifiant

L'attribut **id** permet d'identifier une balise d'un document HTML de manière unique. Deux balises ne doivent pas avoir deux attributs id ayant la même valeur au sein du même document.

Définition dans le code CSS

```
#id_element {
      propriete1: valeur1;
      propriete2: valeur2;
}
```

Définition dans le code HTML

```
<balise id = "id_element">  ……. </balise>
```

Exemple :

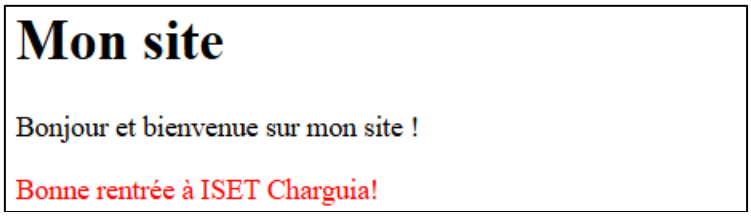
Code dans la feuille de style: monStyle.css:

```
#conclusion {
        color : red;
}
```

Code HTML:

```
<!DOCTYPE html>
<html lang="fr">
  <head>
      <meta charset="utf-8">
      <link rel="stylesheet" href="monStyle.css">
      <title>Test du sélecteur id</title>
  </head>
<body>
      <h1>Mon  site</h1>
      <p>Bonjour et bienvenue sur mon site !</p>
      <p id ="conclusion">Bonne rentrée à  ISET Charguia!</p>
</body>
</html>
```

Résultat :



IV.3. Les sélecteurs de classe

L'attribut **class** permet de nommer un groupe de balises au sein d'un même document HTML. Plusieurs balises peuvent ainsi avoir un attribut **class** ayant la même valeur.

Définition dans le code CSS

```
.non_de_la_classe {
      propriete1: valeur1;
      propriete2: valeur2;
}
```

Définition dans le code HTML

```
<balise class ="non_de_la_classe">  ……. </balise>
```

Exemple :

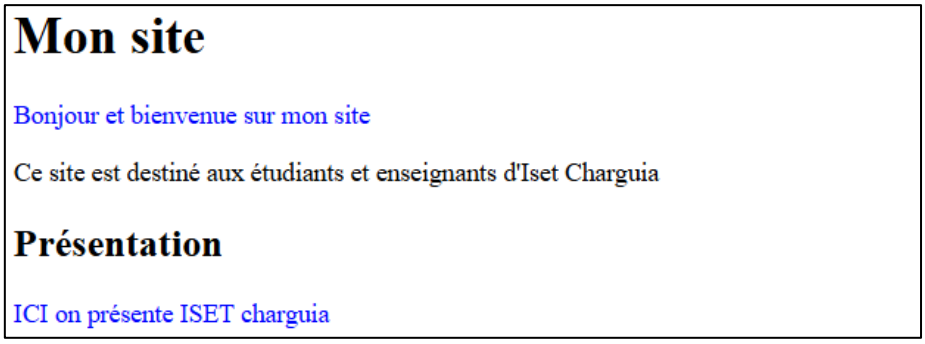
Code dans la feuille de style: MonStyle.css:

```
.introduction {        color : blue;}
```

Code HTML:

```
<!DOCTYPE html>
<html lang="fr">
  <head>
      <meta charset="utf-8" >
      <link rel="stylesheet" href="monStyle.css">
      <title>Test du sélecteur class</title>
  </head>
<body>
<h1>Mon site</h1>
<p class="introduction">Bonjour et bienvenue sur mon site</p>
<p> Ce site est destiné aux étudiants et enseignants d'Iset Charguia </p>
<h2>  Présentation  </h2>
<p class="introduction"> ICI on présente ISET charguia</p>
</body>
</html>
```

Résultat :



IV.4. Application d'un style avec les sélecteurs « span » et « div »

IV.4.1. Balise

… ** : est une balise de type **inline, c'est-à-dire une balise que l'on place au sein d'un paragraphe de texte, pour sélectionner certains mots uniquement auxquels on peut appliquer un style spécifique sans toucher au reste du texte du paragraphe.

Exemple :

Code CSS

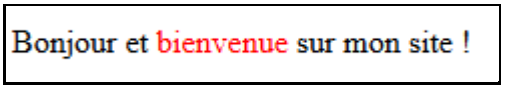
```
.salutations
{       color: red;}
```

Extrait du code HTML

```
<p>Bonjour et <span class="salutations">bienvenue</span> sur mon site !</p>
```

Résultat :

Seul le mot « bienvenue » apparaît en rouge.



IV.4.2. Balise

`<div> …</div>` : est une balise qui permet de marquer une zone

Exemple :

Code CSS :

```
.titre {  color: blue; }
```

Extrait du code HTML

```
<div class="titre">C'est le titre de ma page</div>
```

V. Règles d'application des styles CSS

Le CSS est appliqué selon certaines règles détaillées dans la suite.

V.1. L'héritage

L'héritage des CSS est fondé sur le modèle Parent-Enfant(s) : **chaque élément Enfant reçoit en héritage tous les styles de son élément Parent**. Par exemple, la balise `<html>` est parent de `<body>.`

L'élément enfant hérite de toutes les propriétés de l'élément parent uniquement si ces propriétés s'héritent car l'héritage ne fonctionne pas sur toutes les propriétés css (margin, padding et autres propriétés de bloc ne s'héritent pas).

Exemple :

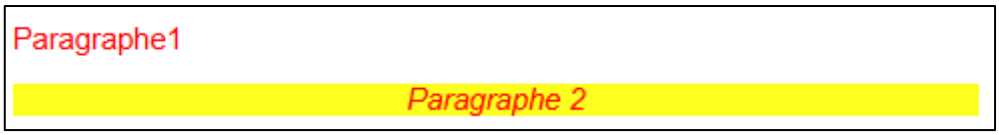
Extrait du code HTML **:**

```
<body class ="stylebody">
<p> Paragraphe1 </p>
<p class="stylep"> Paragraphe 2 </p>
</body>
```

Code CSS:

```
.stylebody {
font-size: 10pt;
font-family: arial, verdana, sans-serif;
color: red;
text-align: left;
background-color: #FFFFFF ;
}
.stylep {
font-style:italic;
text-align:center;
background-color:yellow;
}
```

Résultat :



- Dans le premier paragraphe, les 5 propriétés de la balise `body` s'appliquent
- Dans le 2ème paragraphe :
  - Les **3** propriétés **`font-size, font-family`** et **`color`** de la balise `body` sont gardées
  - Les **2** autres propriétés :
    `text-align: left` et **`background-color: #FFFFFF`**
    ont été remplacées respectivement par
    `text-align:center` et **`background-color:yellow`** qui sont des propriétés propres à lui-même.
  - La propriété **`font-style:italic;`** qui n'a pas été héritée est appliquée

V.2. Cascade de styles

Dans le cas où plusieurs déclarations de styles sont utilisées dans la même page web, les règles CSS sont construites au fur et à mesure de la lecture de chacune d'entre elles. Ainsi si des règles contradictoires apparaissent dans les styles, c'est la **dernière règle** qui sera prise en compte par les navigateurs.

Exemple:

```
/* Définition dans une feuille de style: MonStyle.css */
p {
  color: black;
}
/* Définition dans une feuille de style Couleurs.css */
p {
  color: gray;
}
```

Résultat :

Les textes des paragraphes seront de couleur **grise** si ces styles sont importés de cette manière.

V.3 Règles de priorité des sélecteurs

La cascade ne s'applique que lorsque c'est exactement le même sélecteur qui est employé ou, s'il s'agit d'un sélecteur de priorité supérieure. Ainsi plus le sélecteur d'élément est en quelque sorte *précis* dans sa cible, plus sa priorité est importante.

La précision du sélecteur est appelé **spécificité** dans le jargon CSS. L'ordre des spécificités est le suivant, de la moins prioritaire à la plus prioritaire :

1. Elément Html, (exemple : h1 {color: black})
2. Utilisation d'une classe,
3. Utilisation d'un identifiant
4. Style dans la balise Html,

Exemple

1) Priorité entre classes

Code CSS

```
.c1 {font-style:italic; color:green}
.c2 {color:blue}
```

Extrait du code HTML

```
<h1 class = "c1 c2 "> Le titre est en italique et en bleu. </h1>
```

Résultat :

Le titre est en italique et en bleu.

2) Priorité entre sélecteur balise et classe

Code CSS

```
h1 {font-style:italic; color:green}
.c1 {color:blue}
```

Extrait du code HTML

```
<h1 class = "c1"> Le titre est en italique et en bleu. </h1>
```

Résultat :

Le titre est en italique et en bleu.

3) Priorité entre sélecteur identifiant et classe

Code CSS

```
#id1 {font-style:italic; color:green}
.c1 {color:blue}
```

Extrait du code HTML

```
<h1 id = "id1" class = "c1"> Le titre est en italique et en vert. </h1>
```

Résultat :

Le titre est en italique et en vert.

VI. Les sélecteurs avancés

Outre les sélecteurs de type, d'identifiant et de classe, il existe des sélecteurs avancés : les sélecteurs de combinaison, d'attributs, les pseudo-éléments et les pseudo-classes.

VI.1. Les sélecteurs de combinaison

Il est possible de combiner plusieurs sélecteurs.

Sélecteur	**Rôle**	**Exemple de code CSS**
***** *Sélecteur universel*	Cible toutes les balises sans exception	`* { font-family: Arial; }`
A B *Une balise contenue dans une autre*	Cible toutes les balises **B** situées à l'intérieur d'une balise **A** (les descendants).	`div p { color: red;}` →cible toutes les balises **<p>** enfants situées à l'intérieur d'une ** **
A > B *Une balise enfant direct d'une autre*	Cible toutes les balises **B** enfants directs de **A**	`div>span {font-weight: bold;}` →Cible toutes les balises **** enfants immédiats de la balise ** **.
A + B *Sélecteur de frère adjacent*	Cible la première balise B qui suit immédiatement la balise A	`div+p { background-color: yellow; }` → Sélectionne la première balise **<p>** située après la balise ** **.
A ~ B *Sélecteur de frère général*	Cible tous les éléments **B** qui suivent un élément **A** et qui partagent le même parent,	`div~p { text-decoration: underline; }` → Sélectionne toutes les balises **<p>** situées après une balise ** **.
.c1.c2 (deux classes accolées)	Cible les éléments ayant des 2 classes c1 et c2	`.grand { font-size:24px } .italic{ font-style: italic; } .grand.italic { font-family: 'Times New Roman';}` → La police « Times » sera appliquée aux éléments ayant les classes "grand" et "italic" (au moins)
.c1 .c2 (deux classes qui se suivent)	Cible l'élément descendant ayant la classe c2 du parent ayant la classe c1	`.grand .italic { text-transform: lowercase;}` → La police minuscule sera appliquée aux éléments ayant la classe "italic" descendant (direct ou indirect) de l'élément ayant la classe "grand"

Exemple :

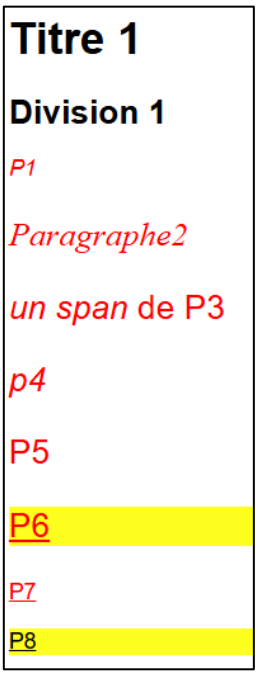
Code CSS :

```
* { font-family: Arial;      }
div p {   color: red;      }
div>span {  font-weight: bold;      }
div+p {  background-color: yellow;      }
div~p {  text-decoration: underline;      }
.grand {   font-size:24px      }
.italic{     font-style: italic;      }
.grand.italic { font-family: 'Times New Roman';}
.grand .italic { text-transform: lowercase;}
```

Extrait du code HTML

```
<h1>Titre 1</h1>
<div>
  <span class="grand" >Division 1</span>
  <p class="italic"> P1 </p>
  <p class="grand italic"> Paragraphe2</p>
  <div class="grand" >
    <p><span class="italic" >
     Un Span </span>de P3</p>
    <p class="italic" >P4</p>
    <p>P5</p>
  </div>
  <p class="grand" >P6</p>
  <p>P7</p>
</div>
<p>P8</p>
```

Résultat:



VI.2. Les sélecteurs d'attributs

Les sélecteurs d'attribut permettent de cibler un élément selon la présence d'un attribut ou selon la valeur donnée d'un attribut.

Sélecteur	Rôle	Exemple de code CSS
A[attribut]	Cible les balises A ayant l'attribut spécifié	`span [class]` → S'applique sur les balises span ayant un attribut class

A[attribut="Val"]	Cible les balises A dont la valeur d'attribut est **exactement** « Val »	`a [title= "Cliquer ici "]`
		`{ color : red ;}`
		→S'applique sur une balise dont l'attribut title a pour valeur « Cliquer ici". Exemple :
		<**a** href="http://site.com" title="Cliquer ici">

A[attribut^="val"]	Cible les balises A dont l'attribut **commence** exactement par "val".	**a[href^="https://"]** → cible tous les liens qui commencent par https://.
A[attribut$="val"]	Cible les balises A dont l'attribut **se termine** exactement par "val"	**img[src$=".png"]** → cible toutes les images dont l'URL se termine par .png.
A[attribut*="Val"]	Cible les balises A dont la valeur d'attribut **contient** « Val »	**a[title*="ici"]** **{ color : red ;}** → S'applique sur une balise : <**a** href="http://site.com" title=" ici ou là">

VI.3. Les pseudo-éléments

Un **pseudo-élément** est un mot-clé ajouté à un sélecteur qui permet de mettre en forme **certaines parties** de l'élément ciblé par la règle.

Syntaxe :

```
sélecteur::pseudo-élément {
      propriété: valeur;
}
```

Tableau 1: Principaux pseudo-éléments

Pseudo-élément	Signification
::first-line	Sélectionne la première ligne (block)
::first-letter	Sélectionne la première lettre (block)
::selection	Sélectionne la partie d'un élément sélectionnée (par un double clic)
::before	Insère du contenu au début d'un élément
::after	Insère du contenu à la fin d'un élément
::marker	Cible et stylise un élément d'une liste

Exemple :

Code CSS

```
p::first-letter {
     font-size: 2em;
     color: red;
     font-weight: bold;
 }
 p::first-line {
     background-color: yellow;
     font-style: italic;
 }
```

Extrait du code HTML

```
<h1>Titre 1</h1>
<p>Paragraphe 1, phrase 1<br>
  phrase 2
</p>
<p>Paragraphe 2</p>
<ul>
     <li>TI</li>
     <li>SEG</li>
 </ul>
```

```
::selection {
    background-color: pink;
}
h1::before {
    content: "*";
    color: green;
}
h1::after {
    content: "!!";
    color: blue;
}
li::marker {
   color: blue;
   font-size: 2em;
   content: "**";
}
```

En sélectionnant le mot Paragraphe, le rendu est :

*Titre 1!!

Paragraphe 1, phrase 1
phrase 2

Paragraphe 2

**TI
**SEG

VI.4. Les pseudo-classes

Une **pseudo-classe** permet de cibler des éléments HTML **dans des états spécifiques** (survol, clic, activation, position dans la structure HTML,…) et d'y appliquer des styles conditionnels.

Syntaxe :

```
sélecteur:pseudo-classe {
      propriété: valeur;
}
```

Tableau 2: Principales pseudo-classes

Pseudo-classe	Signification
Pseudo-classes structurelles	
:first-child	Tout élément qui est le premier élément enfant par rapport à son élément parent
:last-child	Tout élément qui est le dernier élément enfant par rapport à son élément parent
:nth-child(n)	Eléments dont l'indice dans la liste des éléments enfants de leur parent est n
:nth-last-child(n)	Nième élément enfant en partant de la fin
:nth-of-type(n)	Elément qui est le nième frère de son type.
:only-child	Elément qui est le seul enfant de son élément parent.
:empty	Élément sans enfants
Pseudo-classes dynamiques	
:hover	Élément survolé par la souris
:active	Élément activé (pendant le clic)
:focus	Élément ayant le focus clavier
Pseudo-classes de lien	

:visited	Lien déjà visité
:link	Lien non encore visité
Pseudo-classes fonctionnelles	
:not(selector)	Eléments qui ne sont pas représentés par le sélecteur passé en argument
:is(selector)	Eléments correspondant à l'un des sélecteurs
:has(selector)	Élément qui contient le sélecteur passé en argument
Pseudo-classes de formulaire	
:checked	Case à cocher ou radio sélectionnée
:disabled	Élément désactivé
:enabled	Élément activé
:required	Input obligatoire
:valid	Input avec valeur valide
:invalid	Input avec valeur invalide
:read-only	Élément en lecture seule
:read-write	Elément en lecture et écriture
:optional	Eléments lesquels l'attribut required n'est pas activé
:in-range	Champs de saisie dont la valeur est dans l'intervalle autorisé (entre min et max)
:out-of-range	Champs dont la valeur courante est en dehors de l'intervalle défini par les attributs min et max.

Exemple 1:

Code CSS

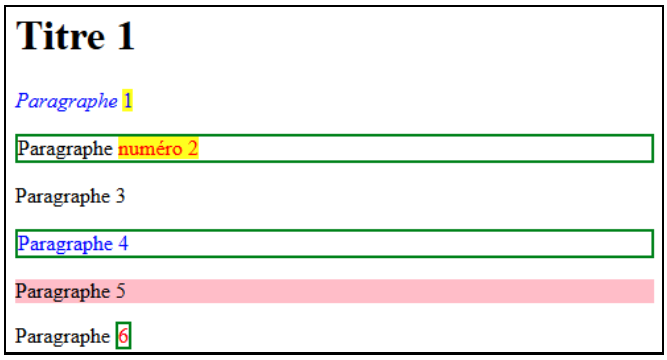
```
.bordure {
   border: 2px solid green;
}
p:hover {
   background-color: pink;
}
span:not(.bordure){
   background-color: yellow;
}
p:nth-child(2){
   color: blue;
}
span:first-child{
   color:red
}
```

Extrait du code HTML

```
      <h1>Titre 1</h1>
  <p><em>Paragraphe</em> <span>1</span></p>
  <p class="bordure">Paragraphe <span>numéro 2</span></p>
  <div>
    <p>Paragraphe 3</p>
    <p class="bordure">Paragraphe 4</p>
    <p>Paragraphe 5</p>
  </div>
  <p>Paragraphe <span class="bordure">6</span></p>
```

Résultat :

Quand le paragraphe 5 est survolé :



Exemple 2:

Cet exemple illustre la mise en œuvre d'une liste horizontale.

Exemple :

Code CSS

```
ul#menu {
   padding: 0;
}

ul#menu li {
   display: inline;
}

ul#menu li a {
   background-color: black;
   color: white;
   padding: 10px 20px;
   text-decoration: none;
   border-radius: 4px 4px 0 0;
}

ul#menu li a:hover {
   background-color: orange;
}
```

Code HTML

```
<h2>Liste Horizontale</h2>
<ul id="menu">
   <li> <a href="python.html">Python</a> </li>
   <li> <a href="c.html">C</a> </li>
   <li> <a href="java.html">Java</a> </li>
   <li> <a href="csharp.html">C#</a> </li>
</ul>
```

Résultat

En survolant le lien C :



Ce chapitre a détaillé comment styliser des éléments individuels et leur donner une apparence attrayante. Cependant, dans le flux normal d'un document Web, les éléments se superposent de haut en bas.

Comment orchestrer la disposition globale d'une page ? C'est précisément l'objet du chapitre 4 : Le positionnement en CSS.